
Autonomous Systems and Mobile Robots

Exercise Sheet 6 – Summer Term 2026

Issued: 09.06.2026 Due: — Discussion: week of 16.06.2026

Professor

Prof. Dr. Malte Schilling
mschilling@uni-muenster.de

Tutor

Janosch Bajorath
j.bajorath@uni-muenster.de

So far your robot has been driving around on the ground — a mobile platform that senses its environment and reacts to it. Today you meet a different kind of robot: a *robotic arm*. The UR3e is a six-axis collaborative industrial robot made by Universal Robots. Six revolute joints connect a kinematic chain from a fixed base to a free end-effector. A ROS 2 package has been prepared for you: it brings up the robot description, joint controllers, Gazebo simulation, and MoveIt 2 in one command.

What you will do: explore the ROS 2 graph of a running arm, command it to a known joint configuration via the terminal, verify the resulting end-effector position using the TF tree, and finally use MoveIt 2's visual planner to move the arm to a goal near an obstacle — without writing a single line of code.

Setup. The arm depends on several upstream ROS 2 packages that are not installed by default. They are listed in `ur3e.repos`, a manifest for `vcs` — a tool that clones multiple Git repositories in one command. Unzip the provided archive, move its contents into a fresh workspace, and let `vcs` fetch the dependencies:

```
1 mkdir -p ~/ur3e_ws/src
2 unzip ur3e-packages.zip
3 mv ur3e-packages/* ~/ur3e_ws/src/
4 cd ~/ur3e_ws
5 vcs import src < src/ur3e.repos
6 git -C src/Universal_Robots_ROS2_Driver apply ../ur_controllers_jazzy_compat.
   patch
7 colcon build --symlink-install
8 source install/setup.bash
```

Then start the full simulation in one terminal and leave it running for all three tasks:

```
1 ros2 launch ur3e_wrapper bringup.launch.py
```

Gazebo and RViz will open. The arm may take a few seconds to initialise its controllers.

Task 1 – Meet the Arm

A prebuilt ROS 2 package has launched the UR3e for you. Open a terminal and start exploring.

(a) **Survey the topic graph.**

Run `ros2 topic list` and `rqt_graph`. Which topics are active? Which node publishes `/joint_states`?

Hint. In `rqt_graph`, set the dropdown to *Nodes/Topics (all)* and uncheck *Hide dead sinks* and *Hide leaf topics* to see the full picture. `ros2 topic info /joint_states` lists publishers and subscribers directly.

(b) **Read the joint state message.**

Run:

```
1 ros2 topic echo /joint_states
```

List all six joint names as they appear in the message. What is the message type?

Hint. `ros2 topic info /joint_states` reports the type. `ros2 interface show sensor_msgs/msg/JointState` shows the full field layout.

(c) **Inspect the TF tree.**

Run:

```
1 ros2 run tf2_tools view_frames
```

This writes a `frames.pdf` file in your current directory. Open it. How many frames are in the kinematic chain from `base_link` to the end-effector? What is the name of the end-effector frame?

Hint. On UR robots the end-effector frame is named `tool0` (ROS-Industrial convention). Count every frame in the chain, including intermediate wrist frames.

(d) **Reflection.**

`robot_state_publisher` publishes the TF tree continuously as the arm moves. What two inputs does it combine to do this, and how do they correspond to the FK calculation you did in the notebook?

Task 2 – Take Manual Control

You will command the arm to a specific joint configuration via the terminal and verify the resulting end-effector pose.

(a) **Find the joint trajectory controller.**

Run `ros2 action list`. Which action server accepts joint trajectory goals?

Hint. Look for an action whose name contains `follow_joint_trajectory`.
`ros2 action info <name>` shows the goal message type.

(b) **Move a single joint.**

Send a goal that moves only the `shoulder_pan_joint` to $+0.5\text{rad}$ (roughly 28°), leaving all other joints at zero:

```
1 ros2 action send_goal /scaled_joint_trajectory_controller/  
  follow_joint_trajectory \  
2 control_msgs/action/FollowJointTrajectory \  
3 "{trajectory: {  
4   joint_names: [shoulder_pan_joint, shoulder_lift_joint,  
5                 elbow_joint, wrist_1_joint,  
6                 wrist_2_joint, wrist_3_joint],  
7   points: [{  
8     positions: [0.5, 0.0, 0.0, 0.0, 0.0, 0.0],  
9     time_from_start: {sec: 3}  
10  }]  
11 }}"
```

Watch the arm move in RViz. Does it reach the goal instantly or follow a smooth trajectory?

(c) **Move to the reference configuration.**

Send a new goal with the following joint angles (in radians):

$$[q_1, q_2, q_3, q_4, q_5, q_6] = [\pi/2, -\pi/3, \pi/2, -\pi/2, -\pi/2, 0] \approx [1.5708, -1.0472, 1.5708, -1.5708,$$

The arm sweeps 90° to the left and folds the elbow upward.

(d) **Verify the end-effector pose.**

With the arm in this configuration, run:

```
1 ros2 run tf2_ros tf2_echo base_link tool0
```

The FK for this configuration predicts the following `tool0` position relative to `base_link`:

$$p = ([-0.128, 0.352, 0.151]) \text{ m}$$

Does the `tf2_echo` output match?

Hint. `tf2_echo` prints a continuous stream; the translation field is the position

vector. Small numerical differences (< 1 mm) are expected due to floating-point representation.

(e) **Exploration.**

What happens if you send a `shoulder_lift_joint` goal of -2.9 rad (near the UR3e's lower joint limit of -3.14 rad)? Does the arm refuse, slow down, or comply?

Task 3 – Let MoveIt Drive

In Task 2 you commanded the arm joint-by-joint. MoveIt 2 inverts this: you specify where you want the end-effector to go, and MoveIt solves the IK and plans a collision-free path to get there.

(a) **Open the Motion Planning panel.**

In RViz, locate the *MotionPlanning* panel on the left. If it is not visible, go to *Panels* → *Add New Panel* → *MotionPlanning*.

You should see two interactive markers at the end-effector: a coloured sphere for position and a ring for orientation.

(b) **Plan and execute a motion.**

Drag the end-effector marker to a new position above the arm's current pose. Click *Plan* — the panel shows the planned joint-space trajectory as a ghost arm. Click *Execute* to run it.

Hint. If planning fails, try a target closer to the current pose. The UR3e has a limited workspace; large jumps may be unreachable or require MoveIt to reject the goal because no IK solution exists.

(c) **Add a collision object.**

In the *Scene Objects* tab of the MotionPlanning panel, click *Add Box* and position a box in front of the arm at roughly the height of the wrist links. Then drag the end-effector goal so it approaches the box from above. Click *Plan*.

- Does MoveIt plan a path through the box, or does it route around it?
- Try placing the goal *inside* the box. What happens?

Hint. The box appears in the RViz 3D view as a coloured obstacle. MoveIt uses the scene geometry for collision checking during planning — any trajectory that would intersect the box is rejected.

(d) **Replanning.**

Click *Plan* three times for the same goal. Does MoveIt always find the same joint-space trajectory, or do the paths differ? Why might a planner produce different paths for the same goal?